

**MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION
TECHNOLOGY**



**SOFTWARE ENGINEERING - COHORT 49
CT239H - M01**

**PROJECT – BASIC TOPIC
SOCIAL POLICY SUPPORT CHATBOT**

SUPERVISOR

Dr. Phan Phương Lan

STUDENT

Cao Tường Hưng B2303873

Cần Thơ, 08 / 2026

ABSTRACT

This project develops an authenticated web-based chatbot that assists Can Tho University students in finding information about tuition fees, scholarships, tuition exemption and reduction, social support, and student loans. The system combines a Retrieval-Augmented Generation architecture with structured tuition lookup and controlled financial tools. Policy documents are converted to Markdown, classified with business metadata, divided into parent and child chunks, embedded locally, and indexed in Qdrant. At query time, a constrained rewriter clarifies only genuinely vague follow-up questions; deterministic intent routing then selects structured data or an appropriate metadata-filtered retrieval lane. Candidate passages are restored from PostgreSQL, reranked by a multilingual cross-encoder, and supplied to Gemini with explicit grounding instructions. Redis provides short-term conversational context, while PostgreSQL stores accounts and persistent history. The implemented application also supports administrator-controlled PDF ingestion with validation and rollback. The resulting design prioritizes Vietnamese semantic retrieval, source separation, auditable calculations, and explicit abstention when supporting evidence is unavailable. Quantitative chatbot accuracy is intentionally reserved for completion after the final evaluation run.

TABLE OF CONTENTS

ABSTRACT.....	i
TABLE OF CONTENTS.....	ii
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	viii
CHAPTER 1: INTRODUCTION.....	1
1.1 PROBLEM STATEMENT	1
1.2 OBJECTIVE	2
1.2.1 General objective	2
1.2.2 Specific objectives	2
1.3 SCOPE AND LIMITATIONS	3
1.3.1 Project scope	3
1.3.2 Limitations	3
CHAPTER 2: THEORETICAL FOUNDATION	4
2.1 RETRIEVAL-AUGMENTED GENERATION.....	4
2.1.1 Retrieval, augmentation, and generation	4
2.1.2 Document preparation and metadata	4
2.1.3 Parent-child chunking.....	5
2.1.4 Vector search and HNSW	5
2.1.5 Cross-encoder reranking.....	6
2.1.6 Grounded prompt construction	6
2.2 EMBEDDING MODEL SELECTION	6
2.2.1 Selected model.....	6
2.2.1.1 Vietnamese-language specialization	7
2.2.1.2 Policy-domain compatibility	7

2.2.1.3 Semantic retrieval capability	7
2.2.1.4 Published model-card evidence	7
2.2.1.5 Efficient bi-encoder operation	7
2.2.1.6 Capacity and storage balance.....	7
2.2.1.7 Local data control	8
2.2.1.8 Technology integration	8
2.2.1.9 Licence and reproducibility	8
2.2.1.10 Fit with chunking	8
2.3 EMBEDDING MODEL TRADE-OFFS	8
2.3.1 Future embedding benchmark	9
2.4 RERANKER MODEL SELECTION	9
2.5 OTHER TECHNOLOGY CHOICES	12
2.5.1 Technology-selection criteria and operational trade-offs	12
CHAPTER 3: APPLICATION RESULTS	13
3.1 SOFTWARE REQUIREMENT SPECIFICATION	13
3.1.1 Overall description.....	13
3.1.1.1 Product perspective.....	13
3.1.1.2 Actors	13
3.1.1.3 Operating environment	14
3.1.1.4 Constraints and assumptions.....	14
3.1.2 Specific requirements	15
3.1.3 Functional requirements	15
3.1.3.1 FR-01 - Account authentication.....	16
3.1.3.2 FR-02 - Question submission	16
3.1.3.3 FR-03 - Conversational context.....	16
3.1.3.4 FR-04 - Safe query rewriting.....	16
3.1.3.5 FR-05 - Intent-aware retrieval	17

3.1.3.6 FR-06 - Structured tuition lookup.....	17
3.1.3.7 FR-07 - Controlled calculations.....	17
3.1.3.8 FR-08 - History management	17
3.1.3.9 FR-09 - Administrator upload.....	17
3.1.4 Nonfunctional requirements	17
3.1.5 Requirements traceability	18
3.2 SOFTWARE DESIGN.....	19
3.2.1 Application architecture.....	19
3.2.2 Data design	21
3.2.2.1 Document metadata	22
3.2.3 Detailed design	22
3.2.3.1 Document ingestion pipeline	22
3.2.3.2 Chat query pipeline.....	23
3.2.3.3 Safe follow-up rewriting.....	24
3.2.3.4 Intent routing and retrieval lanes	24
3.2.3.5 Structured tuition lookup	24
3.2.3.6 Vector retrieval and reranking	24
3.2.3.7 Tool calling and response construction.....	25
3.2.3.8 History persistence.....	25
3.2.4 Failure handling and data consistency	26
3.2.5 User interface design	27
3.3 TESTING.....	32
3.3.1 Test plan	32
3.3.1.1 Representative test cases.....	33
3.3.1.2 Coverage and acceptance procedure.....	34
3.3.2 Results and analysis	35
3.3.2.1 Evaluation method.....	35

3.3.2.2 Reserved interpretation area	36
CHAPTER 4: CONCLUSION	36
4.1 PROJECT SUMMARY	36
4.2 ACHIEVEMENT OF OBJECTIVES	37
4.3 LIMITATIONS	37
4.4 FUTURE WORK.....	38
REFERENCES	39
APPENDICES	40
API ENDPOINT SUMMARY.....	40
TEXTUAL USER GUIDE	40
DEPLOYMENT OVERVIEW	41
PROPOSED EMBEDDING BENCHMARK PROTOCOL	41

LIST OF TABLES

Table 4. Qualitative comparison of retrieval alternatives	9
Table 5. Retrieval and reranking responsibilities	11
Table 6. Technology stack and rationale	12
Table 7. Technology decision risks and controls	Lỗi! Thẻ đánh dấu không được xác định.
Table 8. System actors and access boundaries.....	13
Table 9. Functional requirements.....	15
Table 10. Nonfunctional requirements.....	17
Table 11. Functional-requirement traceability matrix	18
Table 12. Architecture-layer responsibilities	20
Table 13. Data-store mapping	21
Table 14. Business and technical metadata.....	22
Table 15. Important implementation parameters	25
Table 16. Failure boundaries and consistency controls	27
Table 17. Testing strategy	32
Table 18. Automated test inventory	33
Table 19. Representative automated test cases	33
Table 20. Manual acceptance evidence.....	35
Table 21. Chatbot accuracy evaluation results - intentionally blank	35
Table 22. Achievement of project objectives.....	37
Table 23. Recommended future work.....	38
Table 24. Public API endpoint summary	40
Table 25. Deployment dependencies without secret values	41

LIST OF FIGURES

Figure 1. Use cases	14
Figure 2. Application architecture	19
Figure 3. Relational and vector data design.....	21
Figure 4. Document ingestion and rollback pipeline.....	22
Figure 5. Query routing, retrieval, and answer generation	23
Figure 6. Login and registration interface	28
Figure 7. Main chatbot interface.....	28
Figure 8. Conversation history and session controls	29
Figure 9. Chat answer presentation	30
Figure 10. Administrator PDF upload interface	31
Figure 11. Support and policy-topic page.....	32

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CLO	Course Learning Outcome
CTU	Can Tho University
DBMS	Database Management System
ERD	Entity-Relationship Diagram
FR	Functional Requirement
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
NFR	Nonfunctional Requirement
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
SRS	Software Requirement Specification

CHAPTER 1: INTRODUCTION

1.1 PROBLEM STATEMENT

Student financial policy information at a large university is distributed across regulations, annual tuition schedules, scholarship announcements, administrative forms, and external government decisions. A student who asks a practical question rarely knows which document contains the answer or whether a figure represents the actual tuition rate, a statutory ceiling, or the amount used as the basis for exemption and reduction. Documents also differ in publication date, structure, terminology, and target audience. The resulting search task is not merely a keyword lookup: it requires identifying the correct policy domain, academic year, programme, cohort, subject, and eligibility context before extracting an answer.

Conventional website search and manual browsing impose a high cognitive cost. Students must formulate official terminology, open several PDF files, compare tables, and interpret cross-references. Short natural-language questions such as “How much do I pay after a seventy-percent reduction?” combine several facts that may be stored separately. A generic language model can produce fluent explanations but may mix actual tuition with exemption-basis rates, inherit an entity from an unrelated previous answer, or invent a value when the corpus is incomplete. In a student-finance setting, a confident but unsupported number can directly affect planning and trust.

The project therefore treats retrieval correctness, source separation, and controlled calculation as first-class requirements. The chatbot must accept conversational Vietnamese, preserve relevant context without allowing previous generated content to contaminate retrieval, and route questions to the correct information lane. Exact tuition-rate questions should use structured records when possible, while broader policy questions should use semantic retrieval over verified documents. When the evidence is absent, the system must state that it cannot find the information rather than fabricate an answer.

The implementation problem also includes operational concerns. Accounts and full conversation histories require durable storage; recent context requires low-latency access; document ingestion must validate uploads and avoid partial indexes; and

administrative functions must be restricted by role. The solution must remain understandable enough for a student project while demonstrating a complete software-engineering process from requirements and architecture to implementation, testing, and evaluation.

1.2 OBJECTIVE

1.2.1 General objective

The general objective is to design and implement a web-based assistant that provides concise, source-grounded support for Can Tho University student financial policies by combining Vietnamese semantic retrieval, deterministic business rules, controlled calculations, and persistent conversational services.

1.2.2 Specific objectives

- Accept natural Vietnamese questions about tuition, scholarships, tuition exemption and reduction, social support, and student loans.
- Separate policy domains and fee concepts before retrieval so that semantically similar but legally different sources are not mixed.
- Use exact structured lookup for tuition rates that depend on programme, cohort, major, subject, or special rule.
- Retrieve relevant policy passages through a parent-child vector architecture and refine the candidates with cross-encoder reranking.
- Invoke deterministic scholarship and tuition-reduction tools only when the question explicitly requires calculation.
- Maintain short-term conversational context in Redis and durable accounts, sessions, messages, and parent documents in PostgreSQL.
- Permit administrators to upload validated PDF documents and roll back every artifact if ingestion fails.
- Provide a reproducible test and evaluation process while leaving final quantitative chatbot accuracy for the final assessment run.

1.3 SCOPE AND LIMITATIONS

1.3.1 Project scope

The information scope is limited to documents collected for CTU student financial support. The active metadata catalog currently classifies sources into tuition, scholarship, social-support, student-loan, and other domains. The application provides registration, cookie-based authentication, conversational question answering, session management, finance tools, and administrator document upload. The retrieval pipeline processes Vietnamese Markdown derived from PDF documents and uses metadata such as domain, content kind, fee kind, academic year, source, status, index version.

1.3.2 Limitations

The chatbot cannot guarantee an answer beyond the coverage and currency of its active documents. External model and parsing services introduce network, quota, and provider dependencies. The local reranker requires substantial memory and is configured for a constrained GPU.

Table 1. Project scope boundaries

In scope
CTU student-finance and support policies
Vietnamese natural-language questions
Policy retrieval and controlled calculations
Authenticated web use and persistent history
Administrator PDF ingestion

CHAPTER 2: THEORETICAL FOUNDATION

2.1 RETRIEVAL-AUGMENTED GENERATION

2.1.1 Retrieval, augmentation, and generation

Retrieval-Augmented Generation combines an information-retrieval system with a generative model. Instead of asking the language model to answer only from parameters learned during pre-training, the system first retrieves passages from a controlled corpus. Those passages are inserted into a prompt together with instructions and the user question. The language model then generates an answer conditioned on explicit evidence. This separation is useful for institutional policy because documents change more frequently than model weights and must remain auditable.

In this project, retrieval is not a single vector query. The system first decides whether a question can be answered by the structured tuition catalog or whether one or more document lanes are required. A lane represents a business distinction such as scholarship, student loan, social support, actual tuition, exemption basis, or exemption policy. Each Qdrant request includes an active-status condition and, when enabled, business metadata filters. The retrieved child vectors lead to larger parent documents in PostgreSQL, and a reranker selects the most relevant parents before context construction.

Augmentation labels every context block with source and business metadata. The labels explicitly distinguish actual tuition from the basis used to calculate exemption or reduction. Headers extracted from Markdown are also added so that a passage retains its position in the original policy. The prompt supplies a routing-specific answer instruction and requires the model to report missing evidence instead of substituting a semantically similar document. Generation is therefore constrained by both retrieved text and deterministic routing decisions.

2.1.2 Document preparation and metadata

Uploaded PDF documents are validated before processing. The current API accepts safe PDF filenames, checks the media type and PDF signature, enforces a five-mebibyte limit, validates the document class, and requires a consecutive academic-year value where relevant. LlamaParse converts the validated PDF to Markdown. The

conversion preserves headings and tables better than plain text extraction, providing structure for downstream splitting.

The metadata catalog assigns each source a domain, content kind, fee kind, academic year, and active status. Technical metadata includes the source filename, effective date, timestamp, SHA-256 checksum, index version, and ingestion-run identifier. Business metadata controls retrieval; technical metadata supports drift detection, reproducibility, and rollback. This distinction prevents operational fields from being confused with the semantics of a student question.

During ingestion, heading-level metadata and business labels are attached before the second parent split, so every derived parent and child retains the same filterable classification. A table-aware splitter keeps Markdown tables intact whenever possible and repeats header rows when a large table must be divided. This behavior is important because tuition schedules often store meaning in column headings that would otherwise be separated from numeric rows.

2.1.3 Parent-child chunking

A single chunk size cannot simultaneously maximize retrieval resolution and generation context. Small chunks are easier to match semantically, but they may omit the surrounding rule or table heading. Large chunks preserve meaning but may contain several unrelated topics. The project applies a small-to-big strategy: child chunks are embedded for high-resolution vector search, while the associated parent passages are returned to the reranker and large language model.

The parent splitter uses a 2,800-character target with 100-character overlap and prefers paragraph, line, sentence, and word boundaries. Child chunks use 400 characters with 50-character overlap. Smart table handling preserves headers in every table fragment. The overlap reduces boundary loss, while metadata labels make the business category visible to both the embedding model and the generated prompt.

2.1.4 Vector search and HNSW

Dense retrieval maps a question and document chunks into the same vector space. Cosine similarity compares their orientation, which approximates semantic relatedness while reducing the influence of vector magnitude. Qdrant stores 768-dimensional child

vectors and uses a Hierarchical Navigable Small World index. HNSW builds a multi-layer proximity graph that offers approximate nearest-neighbour search with high recall and low query latency. The configured construction parameters balance graph connectivity and indexing cost for the project corpus.

2.1.5 Cross-encoder reranking

A bi-encoder independently represents questions and documents, enabling fast search but losing some token-level interaction. A cross-encoder receives the query and passage together and can evaluate detailed relationships. Running a cross-encoder over the entire corpus would be expensive, so the system uses vector search for a broad candidate set and reranks only the retrieved parents. The current pipeline searches for fifteen candidates and retains six after reranking. A soft temporal tie-break applies only when relevance scores fall within 0.05, ensuring that a clearly more relevant older policy is not displaced solely by date.

2.1.6 Grounded prompt construction

The final prompt contains the user’s original question, recent human and assistant messages, a retrieval-specific instruction, and labeled evidence. The original wording is preserved for natural response generation even when a clarified search query is used for retrieval. Tool results are appended as structured messages and a final model call verbalizes them. If no authoritative data or correctly filtered passages are available, the API returns an explicit no-result response without invoking speculative generation.

2.2 EMBEDDING MODEL SELECTION

2.2.1 Selected model

The implementation uses bkai-foundation-models/vietnamese-bi-encoder through LangChain’s HuggingFaceEmbeddings adapter. The model maps sentences and paragraphs into a 768-dimensional dense vector space and uses a PhoBERT-base-v2 backbone . It is selected for first-stage retrieval because the application corpus and user questions are predominantly Vietnamese and contain formal administrative language.

2.2.1.1 Vietnamese-language specialization

The model is trained for Vietnamese sentence similarity and semantic retrieval. A language-specific encoder is a more direct fit than an English-oriented encoder for questions containing Vietnamese grammar, diacritics, abbreviations, and institutional vocabulary.

2.2.1.2 Policy-domain compatibility

Its training mixture includes Vietnamese-translated MS MARCO and SQuAD together with data from the Legal Text Retrieval Zalo challenge. The latter is relevant to decisions, regulations, eligibility conditions, and administrative passages that resemble the project corpus.

2.2.1.3 Semantic retrieval capability

Dense 768-dimensional representations allow passages to match paraphrased questions even when they do not share exact surface terms. This supports natural student wording that differs from official policy titles.

2.2.1.4 Published model-card evidence

The authors report stronger legal-retrieval results for the fully trained model than for the listed Vietnamese-SBERT and partial-training baselines [2]. These external benchmark figures are evidence about the model, not measurements of this chatbot.

2.2.1.5 Efficient bi-encoder operation

Document chunks are embedded once during ingestion. At runtime only the query is embedded, after which Qdrant performs nearest-neighbour search. This is materially cheaper than joint query-passage scoring over the full corpus.

2.2.1.6 Capacity and storage balance

A 768-dimensional vector offers more representational capacity than common 384-dimensional compact models while remaining practical for the current document collection and local vector store.

2.2.1.7 Local data control

Embedding runs locally. Policy content does not need to be sent to an external embedding API, reducing network dependency, recurring cost, and unnecessary transfer of institutional documents.

2.2.1.8 Technology integration

The model is directly supported by Sentence Transformers and HuggingFaceEmbeddings. Its output dimension matches the configured Qdrant collection and cosine-distance function, so no projection or adapter service is required.

2.2.1.9 Licence and reproducibility

The published Apache-2.0 model, configuration, and weights support reproducible academic implementation and avoid dependence on a closed embedding endpoint.

2.2.1.10 Fit with chunking

The model card lists a maximum sequence length of 256 tokens. The 400-character child chunks reduce truncation risk, and the 50-character overlap protects meaning around boundaries.

Table 1. Embedding-model selection criteria

Criterion	Project requirement	Selected-model fit
Language	Vietnamese questions and documents	Vietnamese-specific PhoBERT backbone
Domain	Administrative and policy retrieval	Includes Vietnamese legal-retrieval training data
Vector size	Useful semantic capacity with local storage	768 dimensions
Deployment	Repeatable local ingestion and query embedding	Open weights and Sentence Transformers support
Context	Short retrieval chunks	Matches 400-character child strategy

2.3 EMBEDDING MODEL TRADE-OFFS

The second-stage ranking model selected for this system is BAAI/bge-reranker-v2-m3, a multilingual cross-encoder derived from the BGE-M3 model family [4], [5]. Unlike the Vietnamese bi-encoder used in the first retrieval stage, the reranker does not

generate reusable document vectors. Instead, it jointly encodes each query–parent–passage pair and produces a relevance score. This joint encoding allows the model to capture fine-grained relationships that may be missed when the query and documents are encoded independently.

Table 1. Qualitative comparison of retrieval alternatives

Candidate	Advantage	Limitation	Decision
Vietnamese Bi-Encoder	Vietnamese-specific; legal-retrieval data; practical 768-D vectors	Shorter context and older than newer multilingual models	Selected for first-stage retrieval
Multilingual MiniLM	Small and fast	Lower vector capacity and weaker Vietnamese policy alignment	Not selected
External embedding API	Managed infrastructure and simple scaling	Cost, network, privacy, and vendor dependence	Not selected

2.3.1 Future embedding benchmark

A future experiment should freeze the corpus and construct query–passage relevance judgments for each policy domain. Every candidate model should receive identical chunks and metadata filters. Recall at k, mean reciprocal rank, normalized discounted cumulative gain, index size, embedding throughput, and end-to-end latency should be measured. The experiment should also include exact-term and paraphrase subsets so that dense retrieval is not rewarded for behavior that a simpler keyword index could provide.

2.4 RERANKER MODEL SELECTION

The second-stage model is BAAI/bge-reranker-v2-m3, a multilingual cross-encoder derived from BGE-M3. Unlike the embedding model, the reranker does not create reusable document vectors. It accepts each query–passage pair and returns a relevance score. This joint encoding captures detailed relationships that independent query and document vectors can miss.

The model was selected based on both architectural requirements and project-specific empirical evaluation. The first-stage Vietnamese bi-encoder and Qdrant retrieve up to 15 candidate parent passages to maximize recall. BGE Reranker v2-m3 then

reorders these candidates, and only the six highest-ranked parent passages are included in the final prompt. This two-stage design provides a practical balance between retrieval speed, ranking precision, prompt length, and context contamination.

Multilingual support is particularly relevant to the dataset used in this project. Although the policy documents are primarily written in Vietnamese, they also contain English scholarship and programme names, technical terminology, abbreviations, banking terminology, and legal document codes. A multilingual reranker can process these mixed-language expressions without requiring separate ranking models.

A controlled retrieval-only comparison was conducted on ten difficult questions selected from tuition exemption, social support, student loan, and scholarship scenarios. Gemini was excluded from both answer generation and evaluation so that the comparison measured only retrieval and reranking quality. A result was considered a Hit@6 only when the final six parent passages contained both the expected source and the specific fact required to answer the question.

BGE Reranker v2-m3 achieved a Hit@6 of 70% and an MRR of 0.65, while GTE Multilingual Reranker Base achieved a Hit@6 of 60% and an MRR of 0.50. BGE therefore retrieved the required parent passage more frequently and generally placed it at a higher rank.

The end-to-end /chat evaluation provided supporting evidence: the BGE configuration obtained 95% accuracy, compared with 94% for the GTE configuration. However, this one-percentage-point difference was not treated as the primary selection criterion because the end-to-end result may also be affected by Gemini answer generation and LLM-as-a-judge variability. The retrieval-only Hit@6 and MRR measurements therefore provided stronger evidence for model selection.

The system is deployed on a GTX 1650 GPU with 4 GB of VRAM. To remain within this hardware constraint, only the top 15 candidates are reranked and each query–passage pair is limited to 512 tokens.

A soft temporal tie-break is applied only when reranker scores differ by no more than 0.05. Semantic relevance therefore remains the primary ranking criterion, while newer documents are preferred only when their relevance scores are nearly equal.

Based on these results, BGE Reranker v2-m3 was selected as the most appropriate practical trade-off for the current application. This conclusion is limited to the project’s Vietnamese student-finance dataset and available hardware; it does not imply that BGE is universally superior to GTE or other reranking models.

Table 4. Compare between BGE and GTE reranker

Criterion	BGE Reranker v2-m3	GTE Multilingual Reranker Base
Retrieval-only questions	10	Query and precomputed child vectors
Hit@6	70%	60%
Mean Reciprocal Rank	0.65	0.5
End-to-end accuracy	95%	94%
Main observation	Better fact-bearing Parent ranking	Faster, but lower retrieval quality
Decision	Selected	Not Selected

Table 2. Retrieval and reranking responsibilities

Stage	Model or method	Input	Output	Primary goal
1	Vietnamese Bi-Encoder + Qdrant	Query and precomputed child vectors	Top 15 candidate parents	Fast recall
2	BGE Reranker v2-m3	Query-parent pairs	Top 6 ordered parents	Fine-grained precision
3	Soft temporal tie-break	Near-equal reranker scores	Stable recency-aware order	Current-policy preference without relevance override

2.5 OTHER TECHNOLOGY CHOICES

Table 3. Technology stack and rationale

Technology	Project role	Selection rationale
FastAPI	Asynchronous web API	Typed request models, dependency injection, and efficient async integration
PostgreSQL	Persistent relational and parent-document storage	Transactions, relationships, indexing, and JSONB metadata
Redis	Recent conversation context	Low-latency list operations and simple session-scoped keys
Qdrant	Vector and payload store	HNSW search, cosine distance, metadata filters, and collection aliases
Gemini	Answer generation and tool calling	Structured tool-call support and Vietnamese response generation
LlamaParse	PDF-to-Markdown conversion	Layout-aware extraction for headings and tables

The architecture deliberately assigns one primary responsibility to each storage system. PostgreSQL is authoritative for users, sessions, full message history, and parent-document content. Redis caches only the recent conversational window. Qdrant stores child vectors and filterable metadata. The structured tuition catalog represents exact monetary facts that should not be diluted by approximate vector similarity. This separation makes failure modes easier to reason about and prevents a single database from being forced into unsuitable workloads.

2.5.1 Technology-selection criteria and operational trade-offs

Technology selection was evaluated against five project constraints: Vietnamese policy content, correctness of monetary answers, modest local deployment resources, the need for reproducible student work, and the requirement to evolve the corpus without retraining a language model. No component is selected in isolation. For example, local embedding reduces recurring API dependency, but the reranker still benefits from a GPU; PostgreSQL provides transactional durability, while Redis is used only where temporary low-latency state is acceptable. The chosen stack therefore distributes responsibilities according to the consequence of data loss or approximation.

The generative and rewriting services are intentionally separated. Query rewriting is a narrow transformation task with a low temperature prompt, strict output validation,

and a safe fallback to the original question. Answer generation has a broader linguistic role but receives server-selected evidence and deterministic tool results. This design limits the authority granted to either model. A provider failure may reduce availability, yet it cannot silently change the structured tuition catalog, document metadata, account records, or calculation formulas.

The three data stores also have different recovery expectations. PostgreSQL backups protect durable identity, conversation, and parent-document state. Qdrant collections can be rebuilt from the retained source files and metadata manifest, although re-embedding has a computational cost. Redis history can be reconstructed from recent PostgreSQL messages after a session is reopened. Treating Redis and Qdrant as rebuildable projections reduces the risk that a cache or index becomes the only copy of important information.

CHAPTER 3: APPLICATION RESULTS

3.1 SOFTWARE REQUIREMENT SPECIFICATION

3.1.1 Overall description

3.1.1.1 Product perspective

The product is a browser-based client served by a FastAPI application. It is not an unrestricted general-purpose chatbot. Its purpose is to answer CTU student-finance questions from an administered corpus and deterministic business data. The backend coordinates authentication, conversation state, intent routing, structured lookup, vector retrieval, reranking, tool calling, persistent history, and document ingestion.

3.1.1.2 Actors

Table 4. System actors and access boundaries

Actor	Responsibilities	Access boundary
Student user	Register, authenticate, ask questions, continue conversations, and manage owned sessions	Only own sessions and messages

Actor	Responsibilities	Access boundary
Administrator	Perform all authenticated user actions and upload classified policy PDFs	Upload endpoint additionally requires persisted admin role
External model service	Rewrite constrained follow-ups, parse PDF content, and generate grounded answers	Receives only task-specific data through configured clients
Storage services	Persist accounts, history, parents, vectors, and recent context	Accessed through backend application

USE CASES

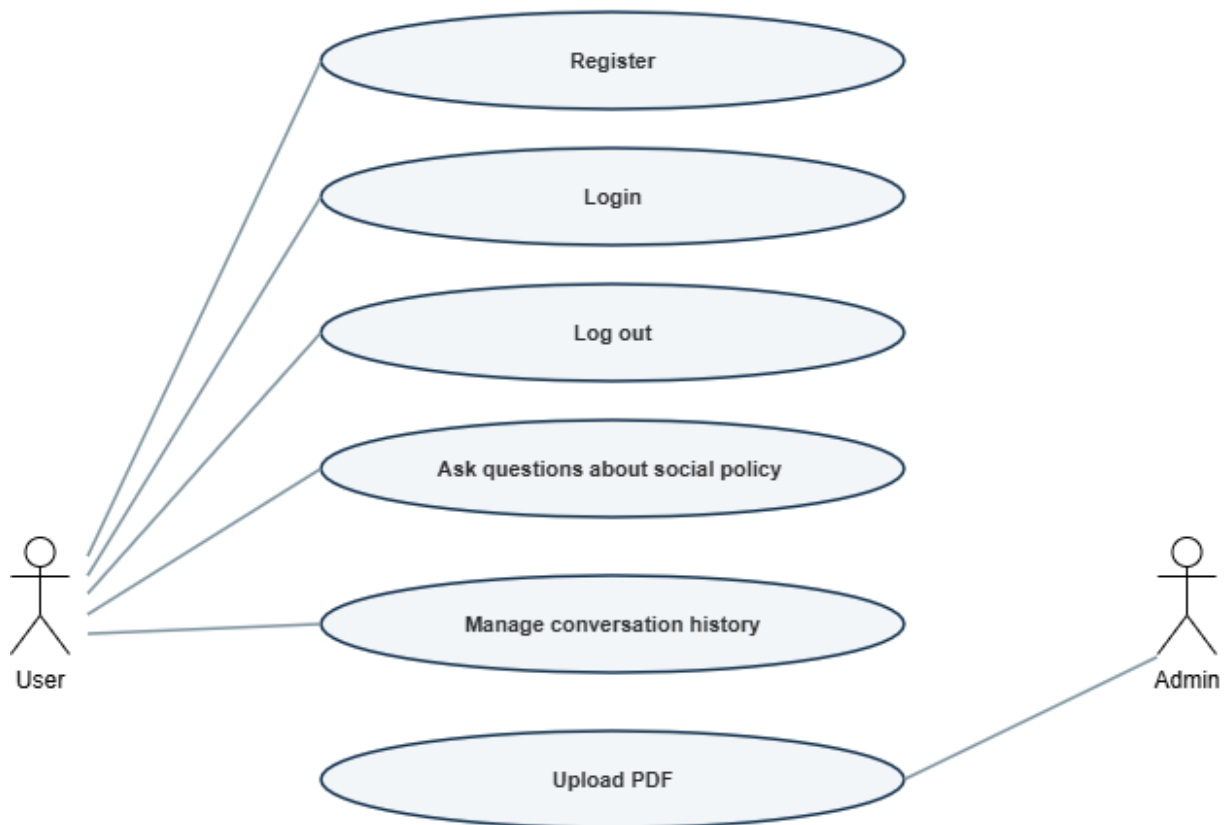


Figure 1. Use cases

3.1.1.3 Operating environment

The application runs as a Python FastAPI service on port 8000 and serves a vanilla JavaScript frontend. PostgreSQL, Redis, and Qdrant are declared as container services. Model execution uses configured Google with local Hugging Face embedding and reranking models. Environment variables hold service URLs and credentials; secrets must not be embedded in source files or documentation.

3.1.1.4 Constraints and assumptions

- A valid Gemini API key is required during application startup.

- Redis, PostgreSQL, and Qdrant must be reachable before chat operations can succeed.
- The active Qdrant collection or alias must match the embedding dimension and metadata schema.
- Uploaded documents must be valid PDF files no larger than five mebibytes.
- A tuition-rate document class requires a consecutive academic-year value.
- The final answer quality depends on document coverage and retrieval configuration.

3.1.2 Specific requirements

The primary user journey begins with registration or login. The server returns an HTTP-only access-token cookie rather than exposing a token to browser JavaScript. After authentication, the user can create or resume a session and submit a question. The server obtains recent context, optionally clarifies a vague follow-up, classifies the question, selects structured or document retrieval, obtains a grounded answer, returns the session identifier, and schedules durable message storage.

The administrator journey extends the authenticated user role. The upload form supplies a PDF, document class, and academic year when required. The server validates the filename, media type, file signature, size, class, and year before writing. It parses and ingests the document using a unique ingestion-run identifier. Any failure after partial ingestion triggers removal of same-run vectors, parents, generated Markdown, copied PDF, and metadata entries.

3.1.3 Functional requirements

Table 5. Functional requirements

ID	Name	Requirement
FR-01	Account authentication	The system shall register users, verify credentials, issue an HTTP-only JWT cookie, report the current user, and delete the cookie on logout.
FR-02	Question submission	The system shall accept an authenticated query and optional session identifier and return an answer with the active session identifier.
FR-03	Conversational context	The system shall load up to five recent Redis messages scoped by user and session.

ID	Name	Requirement
FR-04	Safe query rewriting	The system shall rewrite only vague follow-ups and reject rewrites that introduce unsupported entities, numbers, years, or intentions.
FR-05	Intent-aware retrieval	The system shall classify the question and apply domain, content-kind, fee-kind, academic-year, and active-status filters as appropriate.
FR-06	Structured tuition lookup	The system shall answer exact course, programme, cohort, and special-rule tuition questions from authoritative structured records.
FR-07	Controlled calculations	The system shall invoke scholarship or payable-tuition tools only when the required inputs and user intention justify calculation.
FR-08	History management	The system shall list, load, rename, and delete only the current user's sessions.
FR-09	Administrator upload	The system shall restrict PDF ingestion to authenticated users whose persisted role is admin.

3.1.3.1 FR-01 - Account authentication

The system shall register users, verify credentials, issue an HTTP-only JWT cookie, report the current user, and delete the cookie on logout.

3.1.3.2 FR-02 - Question submission

The system shall accept an authenticated query and optional session identifier and return an answer with the active session identifier.

3.1.3.3 FR-03 - Conversational context

The system shall load recent Redis messages scoped by user and session.

3.1.3.4 FR-04 - Safe query rewriting

The system shall rewrite only vague follow ups and reject rewrites that introduce unsupported entities, numbers, years, or intentions.

The original question remains the source of user intent. The rewritten query is accepted only after validation and is used for retrieval, not as a generated answer. This protects clear questions from inheriting unrelated context and prevents the rewriter from injecting a policy domain or academic year.

3.1.3.5 FR-05 - Intent-aware retrieval

The system shall classify the question and apply domain, content-kind, fee-kind, academic-year, and active-status filters as appropriate.

The route decision may produce more than one lane when the question explicitly combines concepts. Results are deduplicated by document identity. Missing lanes are reported in the answer instruction so that the generator cannot silently substitute another fee type or policy domain.

3.1.3.6 FR-06 - Structured tuition lookup

The system shall answer exact course, programme, cohort, and special-rule tuition questions from authoritative structured records.

3.1.3.7 FR-07 - Controlled calculations

The system shall invoke scholarship or payable-tuition tools only when the required inputs and user intention justify calculation.

3.1.3.8 FR-08 - History management

The system shall list, load, rename, and delete only the current user's sessions.

3.1.3.9 FR-09 - Administrator upload

The system shall restrict PDF ingestion to authenticated users whose persisted role is admin.

3.1.4 Nonfunctional requirements

Table 6. Nonfunctional requirements

ID	Category	Requirement and verification
NFR-01	Security	Passwords shall be bcrypt-hashed; JWTs shall be stored in HTTP-only cookies; admin operations shall verify the persisted role.
NFR-02	Input safety	The upload boundary shall reject traversal names, reserved filenames, invalid media types, forged signatures, and oversized files.
NFR-03	Data integrity	Document metadata mutation and upload rollback shall be atomic and shall not overwrite concurrent additions.
NFR-04	Reliability	The chatbot shall abstain when evidence is unavailable and shall not replace actual tuition with an exemption-basis value.
NFR-05	Performance	Recent history shall use Redis; vector retrieval shall limit candidates; reranking shall process only the candidate set.

ID	Category	Requirement and verification
NFR-06	Maintainability	Authentication, chat, history, document, routing, catalog, RAG, and metadata responsibilities shall remain modular.
NFR-07	Auditability	Retrieved contexts shall retain source, domain, fee kind, year, index version, and ingestion identifiers.
NFR-08	Usability	The browser client shall expose clear authentication, chat, history, and upload states with actionable error messages.

3.1.5 Requirements traceability

Traceability connects each user-visible requirement to an implementation boundary and a verification method. This is especially important in a RAG application because a fluent response alone does not demonstrate that authentication, metadata filtering, structured lookup, rollback, or abstention has worked. The following matrix identifies the principal evidence expected for every functional requirement.

Table 7. Functional-requirement traceability matrix

Requirement	Primary component	Verification focus	Acceptance evidence
FR-01	Authentication API and User model	Registration, cookie login, logout, current user	HTTP status, cookie flags, persisted user
FR-02	Chat API	Authenticated question and session creation	Response schema and owned session ID
FR-03	Redis history service	Recent role-content context and isolation	Scoped key and bounded message list
FR-04	Query rewriter and intent router	No unsupported entity or intent injection	Validated rewrite or original-query fallback
FR-05	RAG engine and metadata lanes	Active, domain, content, fee-kind, and year filters	Retrieved payload and parent metadata
FR-06	Tuition catalog	Exact rate, rule, course, cohort, and clarification behavior	Authoritative structured result
FR-07	Finance tools	Input validation and deterministic arithmetic	Expected numeric output or safe error
FR-08	History API	Ownership, load, rename, and deletion	HTTP result plus PostgreSQL and Redis state
FR-09	Document API	Role, filename, signature, size, class, and year validation	Rejected unsafe upload or accepted work item

Requirement	Primary component	Verification focus	Acceptance evidence
FR-10	Ingestion pipeline	Manifest mutation, indexing, file movement, and rollback	Complete success state or same-run cleanup

Nonfunctional requirements are verified across several functional tests rather than by a single test case. For example, hallucination control is supported by intent routing, metadata filters, catalog authority, no-result behavior, and prompt constraints. Security is supported by authentication dependencies, persisted role checks, ownership filters, cookie settings, and upload validation. This cross-cutting view prevents a single passing endpoint test from being interpreted as proof of the broader quality attribute.

3.2 SOFTWARE DESIGN

3.2.1 Application architecture

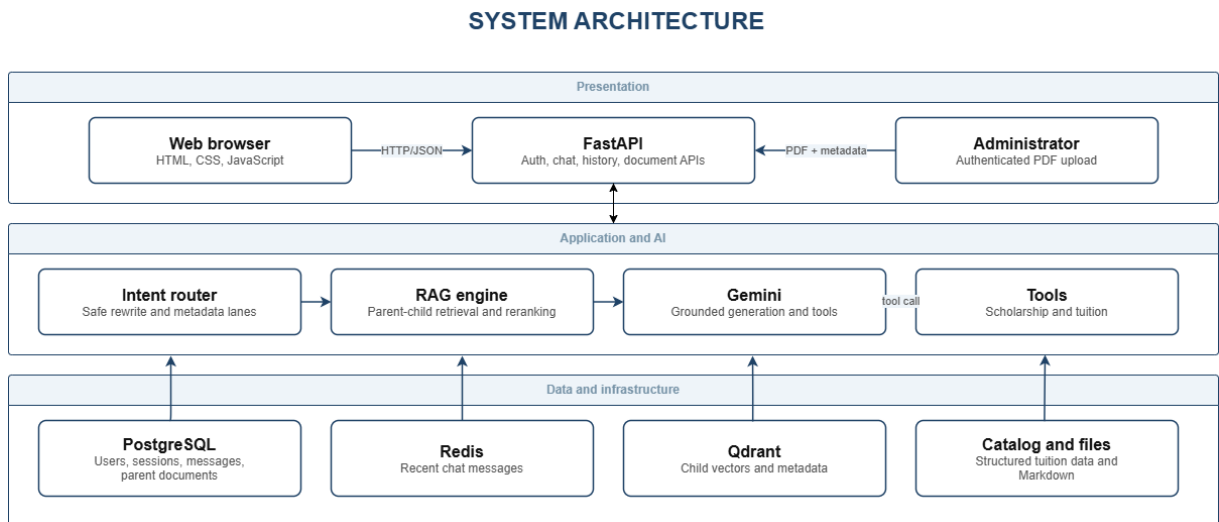


Figure 2. Application architecture

The presentation layer consists of static HTML, CSS, and JavaScript served from the same FastAPI origin. The browser calls authentication, session, chat, and document endpoints with cookies. The application layer coordinates typed request models and authorization dependencies. It separates deterministic intent and catalog logic from RAG retrieval and from the generative model.

The AI orchestration layer uses a low-temperature Groq model only for constrained follow-up rewriting and Gemini for grounded answer generation and tool

calling. The RAG engine is responsible for ingestion, vector retrieval, parent restoration, reranking, and metadata filtering. Scholarship and payable-tuition calculations are deterministic functions, ensuring that arithmetic is not delegated to unconstrained text generation.

The data layer separates workloads. Redis stores recent role-content pairs for low-latency context. PostgreSQL stores users, sessions, messages, and parent documents. Qdrant stores child embeddings and metadata used for filtering. The tuition catalog stores exact rate records and rules. The document metadata manifest ensures that every active Markdown source has a validated business classification.

Table 8. Architecture-layer responsibilities

Layer	Principal components	Responsibility
Presentation	index.html, support.html, script.js, style.css	User interaction, validation feedback, and API calls
API	auth.py, chat.py, history.py, document.py	Request boundaries, authorization, orchestration, and responses
Domain services	query_intent.py, tuition_catalog.py, document_metadata.py	Deterministic classification and business data
AI services	rag_engine.py, ocr_service.py, Gemini and Groq clients	Ingestion, retrieval, reranking, parsing, and generation
Persistence	PostgreSQL, Redis, Qdrant, JSON/Markdown files	Durable data, recent context, vectors, and source artifacts

3.2.2 Data design

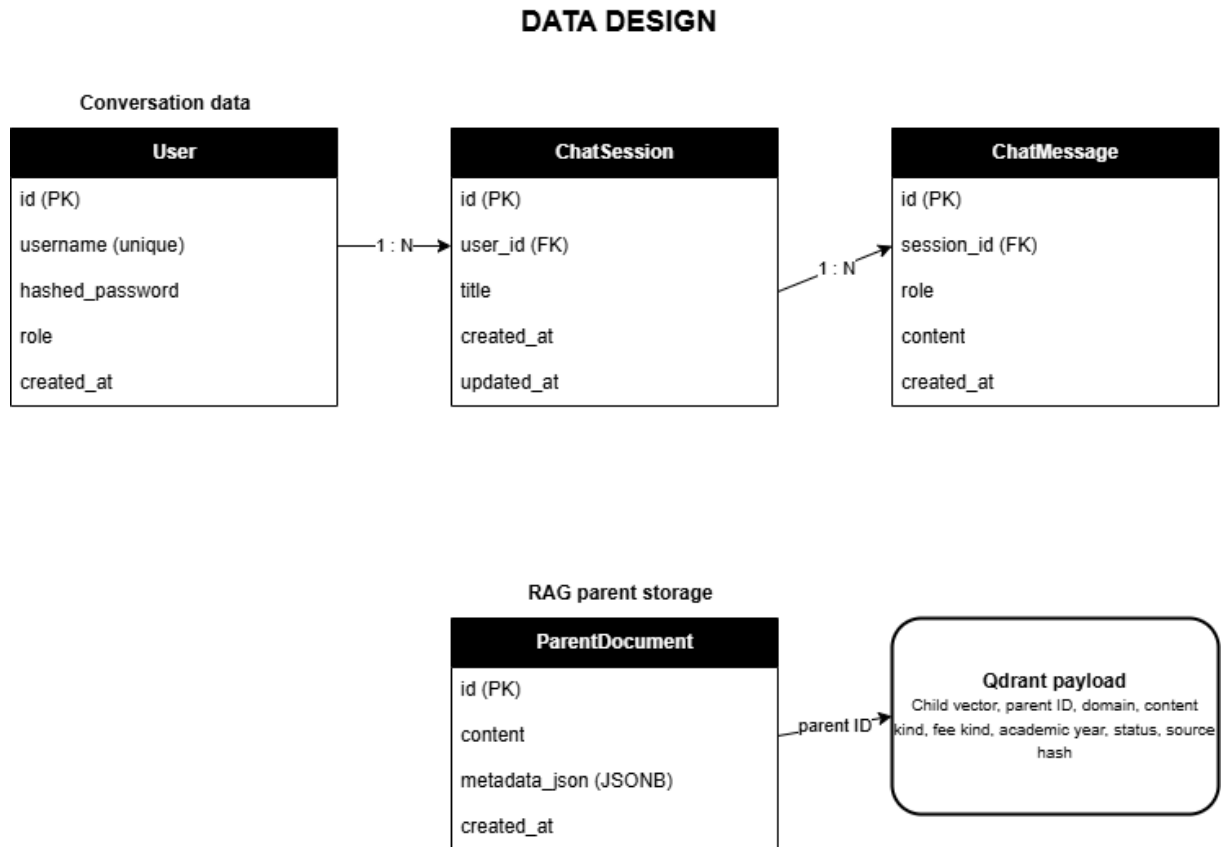


Figure 3. Relational and vector data design

The User-to-ChatSession and ChatSession-to-ChatMessage relationships are one-to-many with delete-orphan cascades. UUID version 7 string identifiers provide time-ordered identifiers without a central integer sequence. Message roles distinguish human and AI entries. Session titles are initially derived from the first query and can be renamed. ParentDocument stores the original parent text and JSONB metadata under the identifier referenced by Qdrant child payloads.

Table 9. Data-store mapping

Store	Key or identity	Stored information	Retention role
PostgreSQL users	user.id	Username, password hash, role, creation time	Durable
PostgreSQL sessions	session.id + user_id	Title and timestamps	Durable
PostgreSQL messages	message.id + session_id	Role, content, timestamp	Durable

Store	Key or identity	Stored information	Retention role
Redis history	user: {user}:session: {session}:history	Recent JSON role-content pairs	Short-term context
PostgreSQL parents	parent document ID	Parent text and JSONB metadata	Durable RAG context
Qdrant children	vector point and parent ID	Embedding and filterable payload	Search index

3.2.2.1 Document metadata

Table 10. Business and technical metadata

Field	Purpose
domain	Separates tuition, scholarship, loan, support, and other sources
content_kind	Distinguishes policies, rate tables, announcements, forms, and procedures
fee kind	Separates actual tuition, exemption basis, and not-applicable records
academic_year	Scopes year-specific material
status	Excludes inactive content
index_version	Identifies immutable vector build
ingest run version	Scopes rollback

3.2.3 Detailed design

3.2.3.1 Document ingestion pipeline

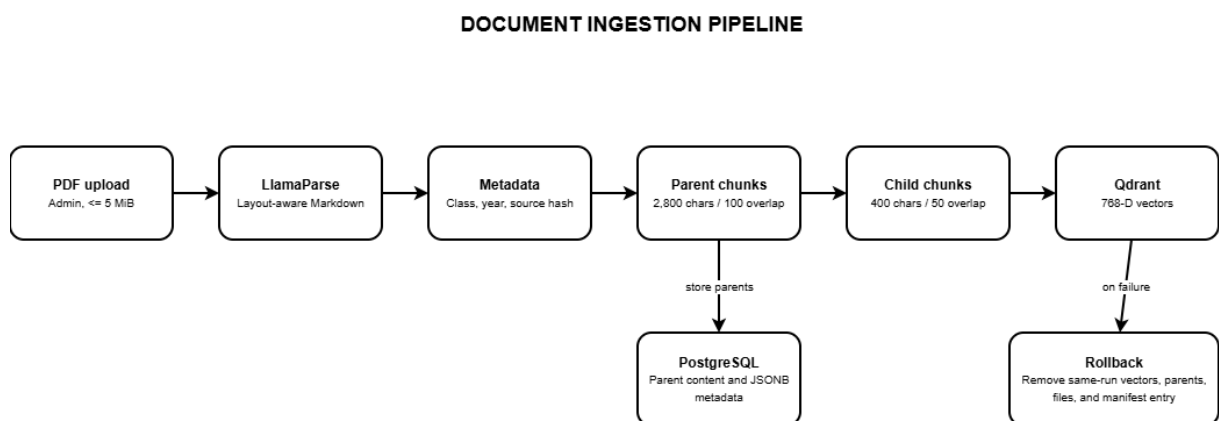


Figure 4. Document ingestion and rollback pipeline

1. Validate the upload boundary before accepting a file path or writing untrusted content.

2. Generate a unique ingestion-run identifier and a collision-safe working filename.
3. Convert the PDF to Markdown through LlamaParse and classify the new source through server-owned mappings.
4. Extract effective date and attach business and technical metadata to heading-derived parent blocks.
5. Preserve tables while splitting large prose parents and generate 400-character child chunks.
6. Embed children with the Vietnamese Bi-Encoder, store vectors in Qdrant, and store parents in PostgreSQL.
7. Move final artifacts and atomically update the metadata manifest only after successful indexing.
8. On any partial failure, purge the same ingestion run and remove generated files and catalog entries.

3.2.3.2 Chat query pipeline

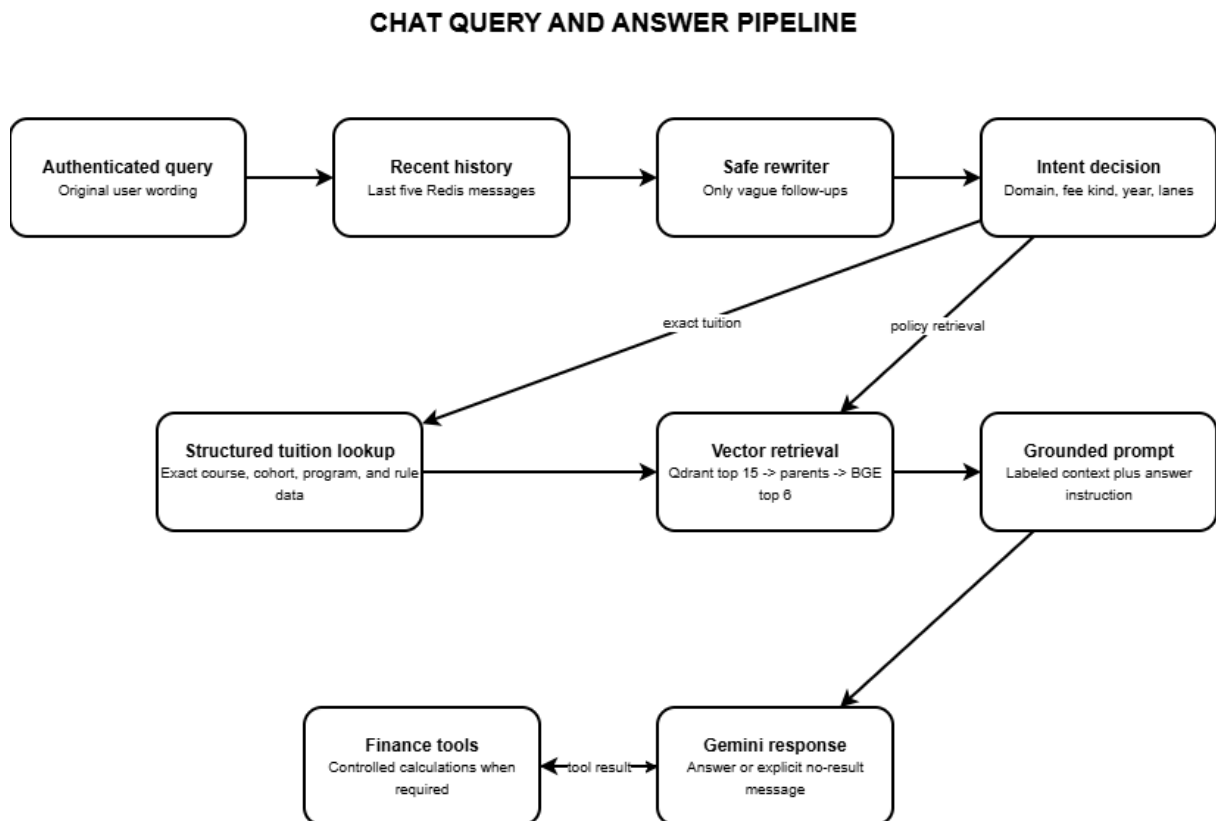


Figure 5. Query routing, retrieval, and answer generation

The API creates a new UUID session when the request does not provide one. Redis history is scoped by both user and session, preventing one account or conversation from

inheriting another user's context. Only the last five entries are supplied to the answer prompt. When a persisted session is loaded, the history endpoint repopulates Redis with the latest stored messages.

3.2.3.3 Safe follow-up rewriting

A heuristic first determines whether the current question is genuinely referential or elliptical. Clear questions bypass the rewriter. For a vague follow-up, the Gemini-3.1-flash-lite model receives only the previous human question and the current question, not the previous AI answer. A validator rejects multiline output, answer-like text, unrelated numbers, new years, new entities, or an intent not supported by the two user questions. Rejection and provider failure both fall back to the original query.

3.2.3.4 Intent routing and retrieval lanes

The classifier recognizes actual tuition, exemption basis, exemption policy, calculation, combined tuition concepts, scholarship, student loan, social support, ambiguous tuition, and other topics. It can extract an academic year and identify whether the route came from the original or clarified query. Each decision maps to immutable retrieval-lane objects containing domain and content constraints. Hard domain separation reduces cross-topic contamination, such as matching SCC scholarship content to SCIC or retrieving loan documents for a tuition question.

3.2.3.5 Structured tuition lookup

Exact tuition questions first enter the structured catalog. The catalog normalizes Vietnamese text and extracts programme, cohort, major, subject, education level, study mode, and special-rule phrases. It can request clarification when an essential dimension is absent. A successful exact lookup is authoritative: the API does not add approximate vector passages that could dilute or contradict the monetary result. A rewritten query is used only when the catalog confirms that the rewrite did not inject an exemption intent or other unsafe entity.

3.2.3.6 Vector retrieval and reranking

For policy questions, the RAG engine creates request-scoped Qdrant filters and retrievers. An active-status condition is always applied. Business metadata filtering can be controlled during rollout so a new collection may be activated safely. The vector store

returns child matches, the ParentDocumentRetriever restores larger PostgreSQL parents, and the BGE cross-encoder scores query-parent pairs. Duplicate parents are removed before prompt construction.

3.2.3.7 Tool calling and response construction

Gemini receives two deterministic tools. The scholarship tool uses GPA, conduct score, and discipline group to apply a defined award schedule. The tuition tool accepts actual tuition, the applicable exemption-basis ceiling, and the reduction percentage; it validates the percentage and calculates the payable amount using the correct cap. If the model requests an unknown tool, the server returns an explicit tool error rather than executing arbitrary code.

3.2.3.8 History persistence

After a response is produced, the API appends human and AI messages to Redis and trims the list to the configured window. A background task creates or updates the PostgreSQL session and writes both messages. History endpoints enforce ownership before listing messages, renaming a session, or deleting it. Deleting a session cascades relational messages and also removes the corresponding Redis key.

Table 11. Important implementation parameters

Parameter	Current value	Design purpose
Parent chunk size / overlap	2,800 / 100 characters	Preserve policy context while controlling length
Child chunk size / overlap	400 / 50 characters	High-resolution semantic matching
Vector dimension	768	Match Vietnamese Bi-Encoder output
Vector distance	Cosine	Compare semantic direction
Qdrant candidate count	15	Broad first-stage recall
Reranker result count	6	Control prompt size and contamination
Reranker max length	512 tokens	Limit GPU memory consumption
Temporal tolerance	0.05	Prefer recent content only among near-equal scores
Redis history window	5 messages	Provide context without excessive prompt growth

3.2.4 Failure handling and data consistency

The chat path distinguishes recoverable absence from infrastructure failure. A correctly processed query with no authoritative catalog result or relevant passage returns an explicit no-result answer. Invalid user input returns a controlled client error. Provider timeouts, database failures, and unexpected exceptions are logged and returned through a generic server boundary without exposing credentials or internal stack traces. This distinction is necessary for evaluation: abstention is a valid safety behavior, whereas an infrastructure error is not a policy answer.

Conversation persistence uses a fast path and a durable path. Redis is updated immediately so the next turn can use recent context. PostgreSQL writes run in a background task to reduce response latency, but ownership and session identity are established before scheduling the write. When a session is reloaded, durable messages repopulate Redis, making the cache disposable. A production deployment should additionally monitor background-task failures because a successful HTTP response does not by itself prove that later persistence completed.

Document ingestion requires stronger atomicity because partial state can contaminate future retrieval. Every upload receives an ingestion-run identifier shared by generated Markdown, parent rows, vector payloads, and metadata entries. The active manifest is changed only after conversion and indexing succeed. If a later step fails, rollback removes records carrying that run identifier rather than deleting by filename alone. This preserves an earlier active version of the same source and avoids erasing concurrent additions.

Index activation is separated from index construction. An immutable Qdrant collection can be populated and checked before an alias points traffic to it. Metadata-filter rollout can likewise be enabled only after payload coverage is validated. These blue-green practices allow a failed migration to leave the previous searchable collection intact. The source checksum and index version then provide enough evidence to determine which document content produced a retrieved passage.

Controlled tools form another consistency boundary. Monetary calculations receive explicit numeric arguments derived from authoritative records or verified user input, validate their allowed ranges, and return structured results. The language model

explains the result but does not choose an undocumented formula. This design makes arithmetic independently testable and prevents a persuasive natural-language answer from hiding a calculation performed on the wrong fee type.

Table 12. Failure boundaries and consistency controls

Failure point	Protected state	Current response	Verification
Query rewrite provider	Original user intent	Reject unsafe output and reuse original query	Injection and provider-error tests
Generation provider	Stored data and tool rules	Return controlled error; do not mutate catalog or index	API failure-path test
Redis unavailable	Durable history	Use PostgreSQL as durable source after recovery	Cache-repopulation integration test
Qdrant retrieval failure	Source and parent records	Surface retrieval failure without fabricated evidence	Retriever error-path test
PDF parse or indexing failure	Previous active document version	Rollback only the same ingestion run	Partial-ingestion test
Background persistence failure	Delivered response and audit trail	Log failure for monitoring and retry design	Injected database-failure test

3.2.5 User interface design

The interface is implemented as a responsive single-page chat experience with modal authentication and administrator upload controls. The following reserved areas are intentionally empty so that final screenshots can be inserted after the deployment appearance has been approved.

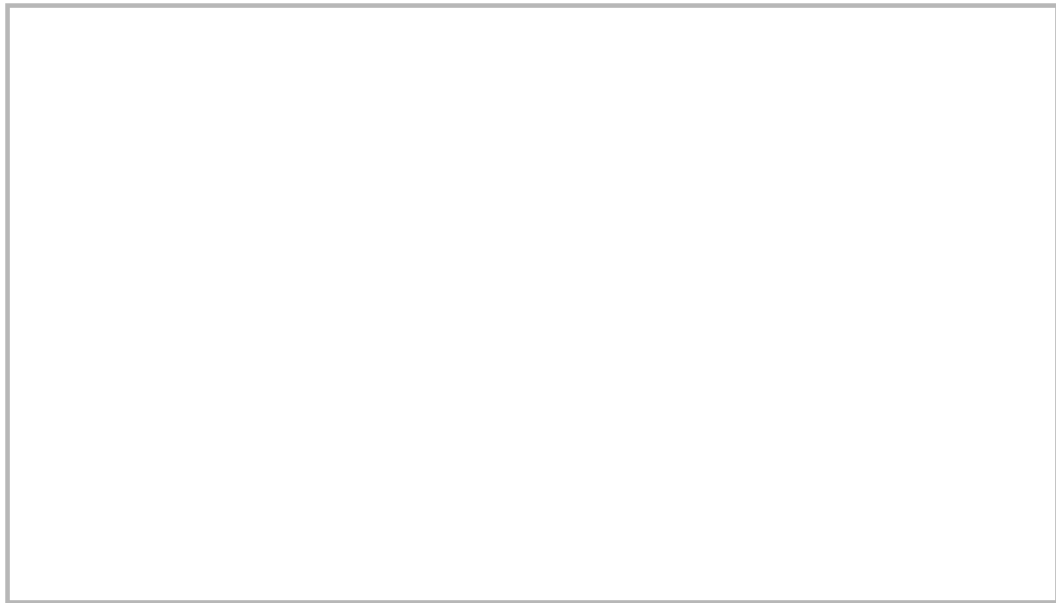


Figure 6. Login and registration interface

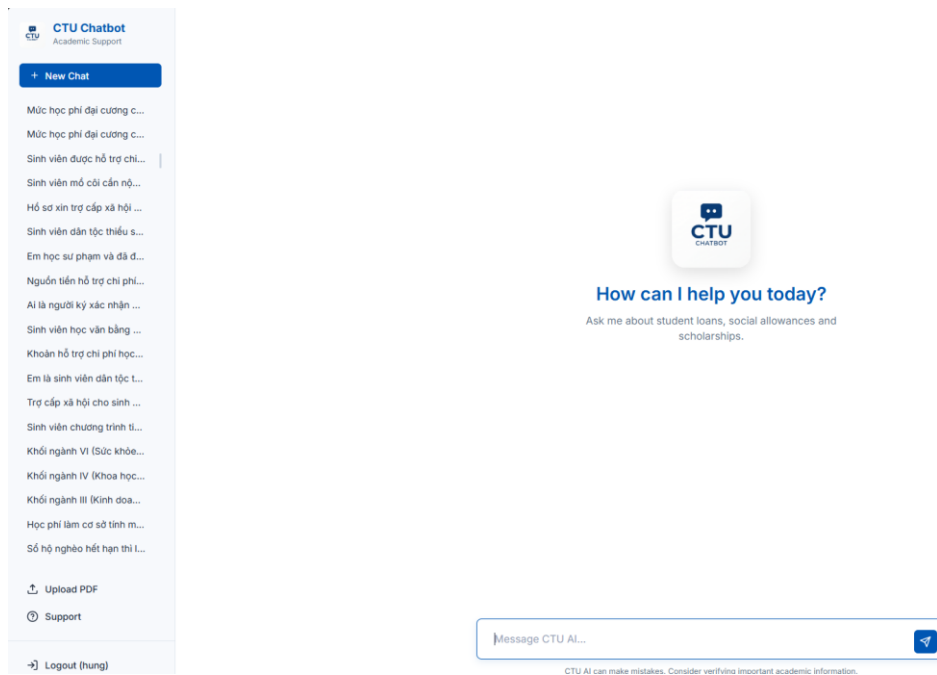


Figure 7. Main chatbot interface

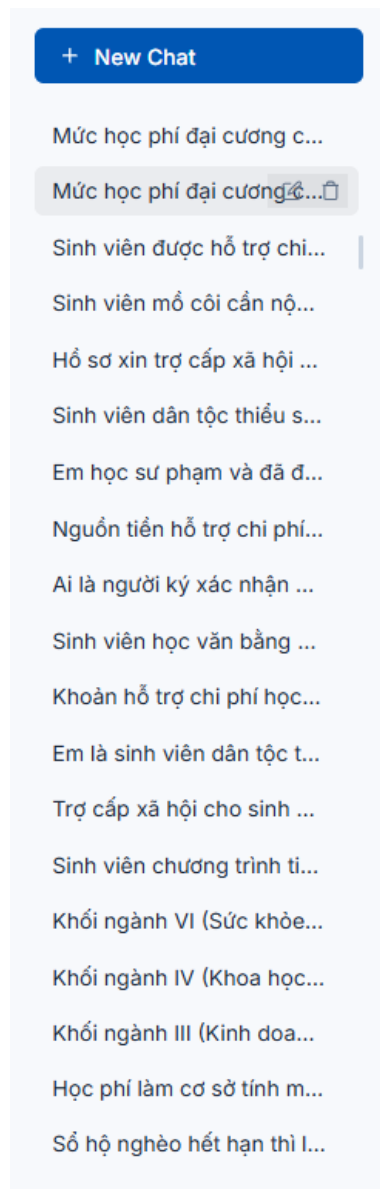


Figure 8. Conversation history and session controls

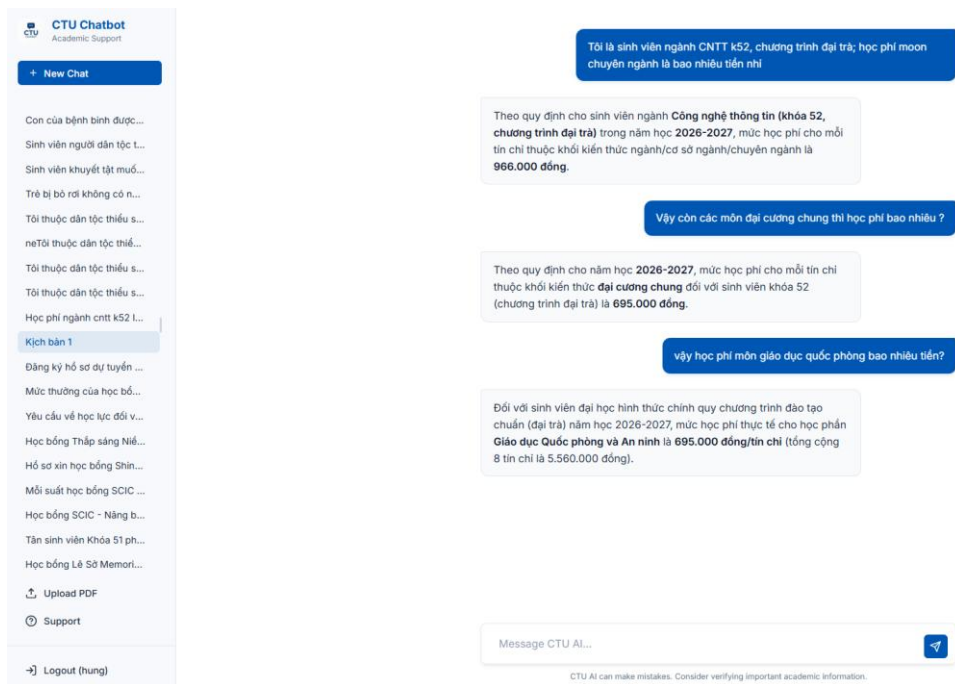


Figure 9. Chat answer presentation

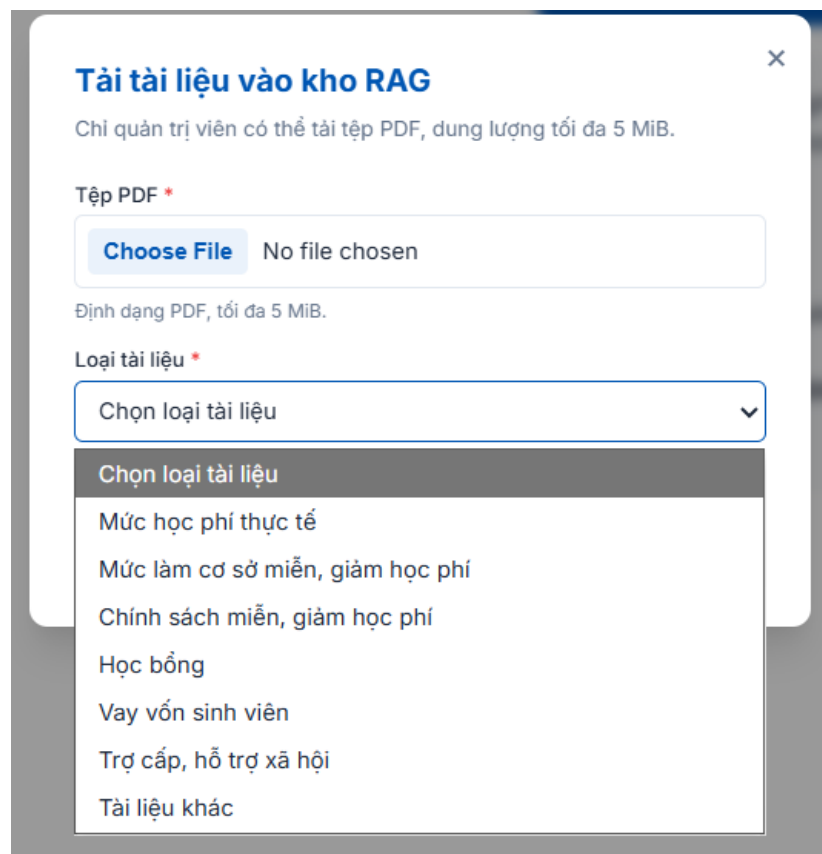
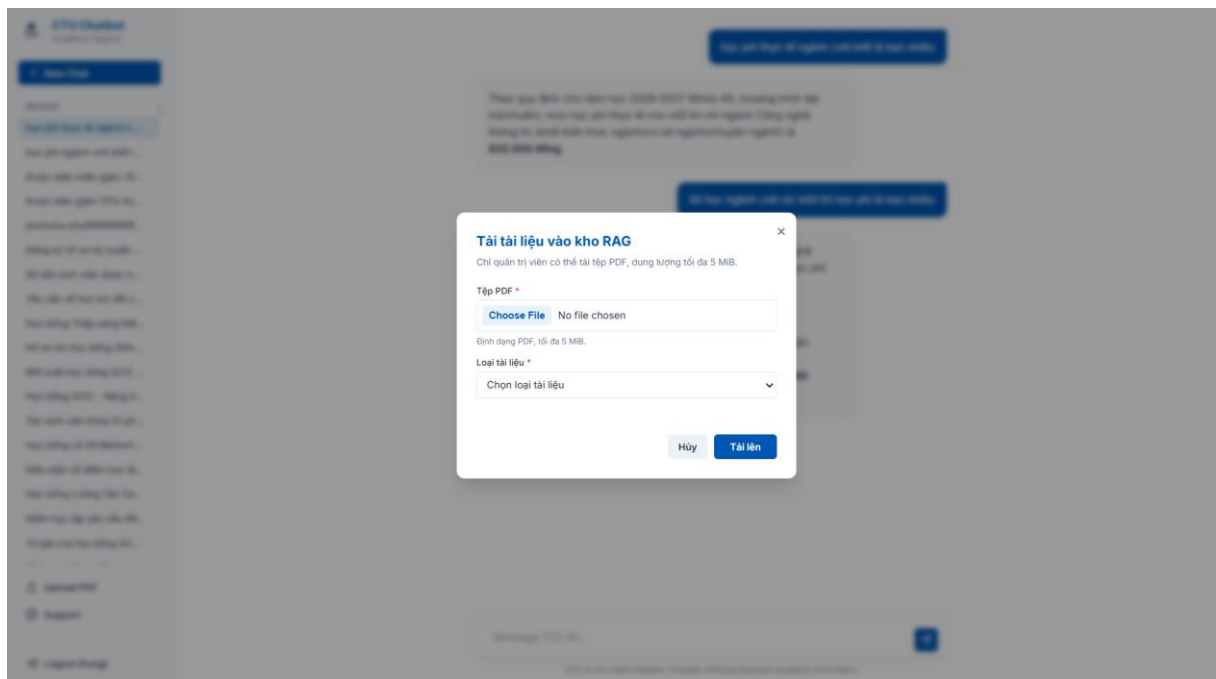


Figure 10. Administrator PDF upload interface

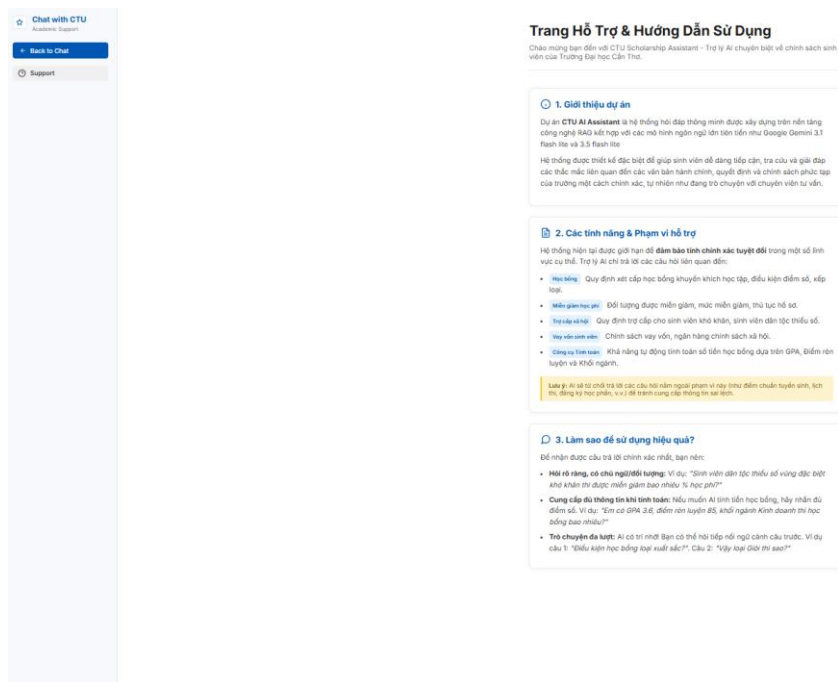


Figure 11. Support and policy-topic page

3.3 TESTING

3.3.1 Test plan

Testing separates deterministic components from external model behavior. Unit tests verify validation, routing, catalog lookup, calculations, and scoring. Integration-style tests exercise the document upload transaction and authorization boundaries using controlled doubles for external systems. System evaluation uses the authenticated HTTP path so that cookies, request schemas, session behavior, and the real /chat orchestration are included.

Table 13. Testing strategy

Test level	Scope	Representative evidence	Pass criterion
Unit	Pure routing, lookup, calculation, and parser behavior	Intent, tuition catalog, finance tools, evaluator tests	Expected value or exception
Integration	API boundary plus coordinated components	Upload, metadata, authorization, rollback	HTTP status and complete state
System	Running browser-equivalent chat flow	Login cookie followed by POST /chat	Valid response and session
Security	Ownership, role, path, file, and token boundaries	Admin and upload validation cases	Unauthorized or unsafe input rejected

Test level	Scope	Representative evidence	Pass criterion
UAT	Student-facing usefulness and clarity	Natural policy questions and manual review	Correct, grounded, understandable answer

Table 14. Automated test inventory

Test file	Cases	Purpose
test_admin_authorization.py	2	Automated source test inventory
test_chat_dataset_eval.py	9	Automated source test inventory
test_document_metadata.py	15	Automated source test inventory
test_document_upload.py	13	Automated source test inventory
test_finance_tools.py	9	Automated source test inventory
test_query_intent.py	41	Automated source test inventory
test_tuition_catalog.py	15	Automated source test inventory
TOTAL	104	Recalculated during report generation

3.3.1.1 Representative test cases

Table 15. Representative automated test cases

ID	Scenario	Expected result	Actual result	Status
TC-01	Student logs in with valid credentials	HTTP-only access cookie is issued	Cookie issued	Pass
TC-02	Non-admin attempts PDF upload	Request rejected with HTTP 403	HTTP 403	Pass
TC-03	Filename contains path traversal	Request rejected before writing	Rejected	Pass
TC-04	Question asks actual tuition while negating exemption	Route to actual tuition	Actual-tuition intent	Pass
TC-05	Course-name tuition lookup omits major	Find course record without false clarification	Record found	Pass
TC-06	Reduction percentage is outside 0-100	Calculation tool rejects input	Validation error	Pass
TC-07	Partial ingestion fails after indexing	Same-run vectors, parents, files, and metadata removed	Rollback completed	Pass
TC-08	Clear new question follows unrelated history	Original intent retained; no inherited entity	Rewrite skipped	Pass

3.3.1.2 Coverage and acceptance procedure

Automated execution should begin with deterministic unit tests because they isolate routing, normalization, calculation, and scoring defects without consuming external quotas. Integration tests then exercise authorization and transaction boundaries with controlled service doubles. A final live run is performed only after the application, corpus, model configuration, and database services are frozen. This sequence makes failures easier to localize and prevents a changing retrieval index from invalidating earlier results.

Authentication acceptance requires more than a successful login response. The test must retain the HTTP-only cookie, confirm that the current-user endpoint resolves the account, verify that protected endpoints reject an unauthenticated client, and demonstrate that one user cannot read or mutate another user's session. Administrator acceptance additionally verifies the persisted role at the upload endpoint; hiding the upload control in the browser is not considered authorization.

Retrieval acceptance uses representative questions from every policy lane. For each question, the reviewer records the classified intent, structured or vector path, applied metadata filters, retrieved sources, and final answer. Exact tuition cases must not fall back to a semantically related exemption-basis document. Scholarship, loan, and social-support cases must remain within their domains. Questions outside the indexed evidence must produce an abstention that does not invent a source or numeric value.

Ingestion acceptance verifies both the successful and interrupted paths. A successful upload must create classified Markdown, parent records, child vectors, metadata entries, and an active searchable version. Injected failure after each major step must leave no artifacts associated with the failed run and must preserve the previous active version. File validation is tested before external parsing so malformed, oversized, forged, or traversal filenames never become parser input.

The 100-question evaluation is the final system measurement, not a substitute for the preceding tests. It uses the same authenticated API flow as the browser and stores expected and actual answers for review. Before reporting accuracy, the evaluator configuration, timestamp, corpus checksum or index version, generator and rewriter

model names, scoring version, and any failed infrastructure calls should be recorded. Blank report cells must remain blank until that evidence is available.

Table 16. Manual acceptance evidence

Acceptance area	Required evidence	Failure interpretation
Authentication and ownership	Cookie flow, current user, cross-user denial	Security failure regardless of chat quality
Routing and catalog	Intent, lane, clarification, authoritative record	Wrong source class or monetary basis
Vector retrieval	Filters, child match, restored parent, reranker order	Recall, filtering, or ranking defect
Finance tools	Validated arguments, formula result, final explanation	Input, arithmetic, or verbalization defect
Ingestion	Complete success artifacts and clean rollback artifacts	Data-integrity or corpus-contamination risk
End-to-end evaluation	Authenticated request log and per-case evidence	Answer defect, abstention, or infrastructure failure

3.3.2 Results and analysis

3.3.2.1 Evaluation method

The chatbot evaluation dataset contains 100 natural student questions distributed equally across four domains: tuition exemption and learning-cost support, tuition, student loans, and scholarships. Each case includes an expected answer and source. The evaluator authenticates through POST /auth/login, retains the HTTP-only cookie, and sends the question to POST /chat. This reproduces the application boundary used by the browser instead of calling internal retrieval functions directly.

The deterministic scorer combines typed numeric recall for money, percentage, dates, durations, and other numbers with Vietnamese content-token recall. Refusal-like answers are penalized when the expected answer contains requested facts. The metric is a repeatable heuristic, not a semantic judge; therefore per-case evidence and manual review remain necessary. The following cells are intentionally blank and will be completed only after the final runtime, corpus, and evaluator configuration have been frozen.

Table 17. Chatbot accuracy evaluation results - intentionally blank

Domain	Passed	Total	Accuracy	Average score
Tuition exemption and learning-cost support				
Tuition				
Student loans				
Scholarships				
Overall				

3.3.2.2 Reserved interpretation area

CHAPTER 4: CONCLUSION

4.1 PROJECT SUMMARY

The project delivers a complete web application for CTU student-finance policy support. It integrates secure account access, persistent conversations, structured tuition lookup, policy-domain routing, Vietnamese semantic retrieval, parent-child context recovery, multilingual reranking, grounded answer generation, controlled finance tools, and administrator document ingestion. The design distinguishes information retrieval from generation and distinguishes approximate semantic retrieval from exact monetary data.

The implementation addresses the central risk identified in Chapter 1: a fluent answer is not sufficient when several similar policy concepts contain different monetary values. Deterministic intent routing and business metadata separate actual tuition, exemption basis, exemption policy, scholarships, loans, and social support. Source labels, no-result behavior, and tool validation make the response process more auditable than unconstrained prompting.

The software-engineering objectives are also represented in the architecture. PostgreSQL, Redis, and Qdrant have separate storage roles; APIs enforce

authentication, ownership, and administration boundaries; upload validation protects the file boundary; and ingestion-run identifiers support rollback. The automated tests cover routing, structured lookup, metadata integrity, finance tools, authorization, file validation, and evaluator behavior.

4.2 ACHIEVEMENT OF OBJECTIVES

Table 18. Achievement of project objectives

Objective	Implemented outcome	Status
Grounded policy answers	Labeled RAG context and explicit abstention instruction	Implemented
Vietnamese semantic retrieval	Vietnamese Bi-Encoder, parent-child retrieval, and BGE reranking	Implemented
Accurate tuition separation	Intent lanes, metadata filters, and structured tuition catalog	Implemented
Controlled calculations	Scholarship and payable-tuition tools with validation	Implemented
Conversation support	Redis recent context and PostgreSQL durable history	Implemented
Safe document growth	Administrator upload, validation, metadata, and rollback	Implemented

4.3 LIMITATIONS

The chatbot remains bounded by the active corpus. Missing, outdated, misclassified, or poorly parsed documents can cause abstention or retrieval errors. External Gemini, Groq, and LlamaParse services introduce availability, quota, and version dependencies. Local embeddings avoid an external embedding API but still require model download and compute resources. The reranker is substantially larger than the embedding model and is constrained by the available GPU and its configured 512-token input.

The current embedding selection is supported by Vietnamese and legal-retrieval evidence, but it has not yet been compared with newer multilingual alternatives on the exact CTU corpus. The evaluator provides deterministic repeatability but cannot fully judge semantic correctness, source faithfulness, or student usefulness. Final quantitative results are intentionally absent from this report version and must be completed after a fresh authenticated evaluation run.

4.4 FUTURE WORK

Table 19. Recommended future work

Priority	Improvement	Expected benefit
High	Run a controlled embedding benchmark and hybrid BM25-dense retrieval experiment	Evidence-based model and retrieval selection
High	Display source citations and policy-effective dates in answers	Improved auditability and user trust
High	Complete final 100-case evaluation and manual error analysis	Measured accuracy and targeted remediation
Medium	Add administration for document status, version, and rollback visibility	Safer corpus operations
Medium	Add latency, retrieval-lane, and no-result monitoring	Operational diagnosis
Medium	Using UAT with students and staff	Usability evidence
Low	Evaluate multilingual and mobile-client support	Broader access

The highest-priority next step is not indiscriminate model replacement. The system should first freeze a representative corpus, construct relevance judgments, and measure candidate retrieval and end-to-end behavior under identical conditions. This approach preserves the evidence-first principle used throughout the project and ensures that added complexity is justified by measurable improvement.

REFERENCES

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [2] BKAI-HUST Foundation Models Lab, “bkai-foundation-models/vietnamese-bi-encoder,” Hugging Face model card. <https://huggingface.co/bkai-foundation-models/vietnamese-bi-encoder>.
- [3] N. Q. Duc, L. H. Son, N. D. Nhan, N. D. N. Minh, L. T. Huong, and D. V. Sang, “Towards Comprehensive Vietnamese Retrieval-Augmented Generation and Large Language Models,” *arXiv:2403.01616*, 2024.
- [4] Beijing Academy of Artificial Intelligence, “BAAI/bge-reranker-v2-m3,” Hugging Face model card. [Online]. Available: <https://huggingface.co/BAAI/bge-reranker-v2-m3>. [Accessed: Jul. 23, 2026].
- [5] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation,” *arXiv:2402.03216*, 2024.
- [6] C. Li, Z. Liu, S. Xiao, and Y. Shao, “Making Large Language Models a Better Foundation for Dense Retrieval,” *arXiv:2312.15503*, 2023.
- [7] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982-3992.
- [8] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE TPAMI*, vol. 42, no. 4, pp. 824-836, 2020.
- [9] Qdrant, “Qdrant Documentation.” [Online]. Available: <https://qdrant.tech/documentation/>. [Accessed: Jul. 23, 2026].
- [11] PostgreSQL Global Development Group, “PostgreSQL 15 Documentation.” [Online]. Available: <https://www.postgresql.org/docs/15/>. [Accessed: Jul. 23, 2026].
- [12] Redis, “Redis Documentation.” [Online]. Available: <https://redis.io/docs/latest/>. [Accessed: Jul. 23, 2026].
- [13] LangChain, “LangChain Documentation.” [Online]. Available: <https://python.langchain.com/docs/>. [Accessed: Jul. 23, 2026].
- [14] Google, “Gemini API Documentation.” [Online]. Available: <https://ai.google.dev/gemini-api>. [Accessed: Jul. 23, 2026].
- [16] LlamaIndex, “LlamaParse Documentation.” [Online]. Available: <https://developers.llamaindex.ai/python/cloud/llamaparse/>. [Accessed: Jul. 23, 2026].

APPENDICES

API ENDPOINT SUMMARY

Table 20. Public API endpoint summary

Method	Path	Authorization	Purpose
POST	/auth/register	Public	Create a student account
POST	/auth/login	Public	Issue HTTP-only access cookie
POST	/auth/logout	Public	Delete access cookie
GET	/auth/me	User	Return current identity and role
POST	/chat	User	Answer a question in a new or existing session
GET	/sessions	User	List owned sessions
GET	/sessions/{id}/messages	Owner	Load messages and repopulate recent history
PUT	/sessions/{id}	Owner	Rename a session
DELETE	/sessions/{id}	Owner	Delete a session and Redis history
POST	/document/upload	Admin	Validate, parse, classify, and ingest a PDF

TEXTUAL USER GUIDE

9. Open the application at the configured FastAPI address.
10. Choose Log in or Register and submit a username and password.
11. After authentication, type a Vietnamese policy question in the message field.
12. Review the answer and provide missing programme, cohort, course, year, or eligibility details when requested.
13. Use the Recent list to reopen a conversation; use the session menu to rename or delete it.
14. For administrator ingestion, open Upload PDF, choose a valid document class, provide a consecutive academic year when required, and submit a PDF no larger than five mebibytes.

The screenshot positions in the main report are intentionally blank. Insert the final deployment screenshots only after checking that no personal account data, access tokens, secret values, or private policy drafts are visible.

DEPLOYMENT OVERVIEW

Table 21. Deployment dependencies without secret values

Component	Default endpoint or mechanism	Required configuration
FastAPI	http://localhost:8000	Python environment and application variables
Qdrant	http://localhost:6333	Collection or active alias
Redis	Configured service endpoint	REDIS_URL
PostgreSQL	localhost:5432/ctu_chatbot	DATABASE_URL
Gemini	External API	GOOGLE_API_KEY
Groq	External API	GROQ_API_KEY
LlamaParse	External API	LLAMA_CLOUD_API_KEY
JWT	HTTP-only cookie	JWT_SECRET_KEY

PROPOSED EMBEDDING BENCHMARK PROTOCOL

15. Freeze the Markdown corpus, metadata manifest, and Qdrant filtering rules.
16. Create domain-balanced Vietnamese questions with passage-level relevance judgments.
17. Use identical parent and child chunks for every embedding candidate.
18. Build a separate immutable Qdrant collection for each model and record vector dimension, size, and build time.
19. Measure Recall@5, Recall@15, MRR@10, nDCG@10, query-embedding time, retrieval latency, and storage cost.
20. Repeat end-to-end evaluation with and without the BGE reranker to isolate first-stage quality.
21. Inspect failures for exact terms, paraphrases, tables, negations, years, named scholarships, and fee-kind confusion.

22. Select a replacement only if the measured gain justifies migration and reindexing cost.